

METROPOLITAN UNIVERSITY
Department of Computer Science and Engineering

PROJECT REPORT

MiniLang Compiler

Submitted by:	Al Mahmud
Course:	Compiler Construction Lab
Department:	Computer Science and Engineering
Institution:	Metropolitan University, Bangladesh

Abstract

This project implements a complete compiler front-end with intermediate-code generation for a small statically typed language named MiniLang. The compiler uses Flex for lexical analysis, Bison for syntax analysis, and C++17 for the Abstract Syntax Tree (AST), nested-scope symbol table, semantic analyzer, diagnostic system, and Three Address Code (TAC) generator. The implementation supports integer, floating-point, and Boolean values; declarations; assignments; expressions; nested blocks; `if`; `if-else`; `while`; and `print`.

The compiler reports lexical, syntax, and semantic errors with line and column locations and, where possible, corrective hints. Parsing and semantic analysis are designed to collect multiple independent errors instead of stopping after the first recoverable problem. Successful programs can be inspected through token, AST, symbol-table, and TAC command-line modes.

An optional Python/Tkinter desktop application was also developed. It uses the existing `build/mcc` executable as its compiler backend and provides a desktop editor, test explorer, structured compiler views, clickable diagnostics, build integration, and a live regression dashboard. The GUI does not duplicate any compiler rules.

The final automated suite contains 42 checks over 32 distinct MiniLang source programs: 15 valid-compilation checks, 17 exact diagnostic checks, and 10 TAC golden-output comparisons. The verified result is 42 passed and 0 failed.

Table of Contents

Abstract	2
1. Introduction	6
1.1 Background.....	6
1.2 Project overview.....	6
1.3 Scope.....	6
1.4 Main outcomes	6
2. Objectives	8
2.1 General objective.....	8
2.2 Specific objectives.....	8
2.3 Acceptance criteria.....	8
3. Language Specification	9
3.1 Data types.....	9
3.2 Lexical elements.....	9
3.3 Operators	9
3.4 Statements.....	9
3.5 Context-Free Grammar	10
3.6 Precedence and associativity	10
3.7 Semantic rules.....	11
3.8 Sample program	11
4. Compiler Architecture	13
4.1 Pipeline.....	13
4.2 Module organization.....	14
4.3 Architectural patterns.....	14
Visitor pattern	14
Collecting diagnostics	14
Annotated AST	14
Scoped hash tables.....	14
4.4 Command-line interface.....	14
4.5 Desktop interface.....	15
4.6 Repository structure	16
5. Lexer Design	17
5.1 Responsibilities	17
5.2 Token categories.....	17
5.3 Longest match and rule ordering	17

5.4 Location tracking	17
5.5 Comments.....	18
5.6 Malformed token handling.....	18
5.7 Token inspection mode.....	18
6. Parser Design	19
6.1 Bison integration	19
6.2 Expression precedence	19
6.3 Dangling else.....	19
6.4 Conflict checking	19
6.5 Error messages	19
6.6 Recovery.....	19
6.7 AST construction.....	20
7. Abstract Syntax Tree	21
7.1 Hierarchy	21
7.2 Ownership.....	21
7.3 Visitor interface.....	21
7.4 AST output.....	22
8. Symbol Table	23
8.1 Symbol record.....	23
8.2 Scope representation.....	23
8.3 Operations.....	23
8.4 Shadowing and visibility.....	24
8.5 Display	24
9. Semantic Analysis	25
9.1 Analysis process.....	25
9.2 Detected errors	26
9.3 Assignment compatibility.....	26
9.4 Multi-error behavior.....	26
9.5 Suppression of later phases.....	26
10. Intermediate Code Generation	27
10.1 Purpose	27
10.2 Instruction forms.....	27
10.3 Expressions.....	27
10.4 Assignment widening.....	27
10.5 Branches	27
10.6 Loops.....	28
10.7 Short-circuit logical expressions.....	28
10.8 Complete sample output	29

11. Challenges	30
11.1 Grammar conflicts and recovery.....	30
11.2 Precise locations across Flex and Bison.....	30
11.3 Malformed-number recovery.....	30
11.4 AST memory ownership.....	30
11.5 Header dependency tracking.....	30
11.6 Cross-platform golden outputs	30
11.7 Responsive GUI execution.....	30
11.8 Structured output without changing the compiler	30
12. Testing	31
12.1 Testing strategy	31
12.2 Accurate test inventory	31
12.3 Coverage.....	32
12.4 Test commands.....	33
12.5 Verified results.....	33
12.6 Reproducibility and repository cleanliness.....	33
13. Conclusion	34
13.1 Achievement summary	34
13.2 Learning outcomes.....	34
13.3 Limitations.....	34
13.4 Future work.....	35
13.5 Final result.....	35
14. References	36
Appendix A - Build, CLI, and GUI Command Reference	37
Appendix B - Complete Automated Check Inventory	38
Appendix C - Compiler Studio Keyboard Shortcuts	39

1. Introduction

1.1 Background

A compiler translates a program from a source language into another representation while checking whether the program follows the language's lexical, grammatical, and semantic rules. Modern production compilers contain many optimization and machine-code stages, but their front ends still rely on the same foundations studied in a compiler-construction course: scanning, parsing, syntax-tree construction, identifier management, type checking, and intermediate representation generation.

MiniLang was selected as a teaching language because it is small enough to implement completely while still exercising the most important front-end problems. It contains multiple data types, operator precedence, nested scopes, control flow, type constraints, and structured intermediate code.

1.2 Project overview

SOURCE-TO-TAC PIPELINE



Figure 1. The six conceptual compiler phases implemented by MiniLang.

The implemented system accepts a MiniLang source file and processes it through six conceptual phases:

1. lexical analysis;
2. syntax analysis;
3. AST construction;
4. symbol-table management;
5. semantic analysis; and
6. TAC generation.

Flex groups source characters into tokens. Bison validates the token sequence and constructs the AST. A semantic visitor enters declarations into a scoped symbol table, resolves identifier uses, and annotates expression nodes with types. The TAC visitor then converts a semantically valid AST into a linear intermediate representation using temporary variables, labels, and jumps.

1.3 Scope

The project implements the complete base language required by the course manual. TAC is the final compiler output. Assembly generation, register allocation, machine-code generation, arrays, functions, and optimization are not claimed as implemented features.

The Tkinter interface is an additional presentation layer. Its purpose is to make the compiler easier to demonstrate and inspect, not to replace or alter the C++ compiler.

1.4 Main outcomes

The final project provides:

- one `mcc` command-line compiler;
 - exact line-and-column diagnostics;
 - a printable AST;
 - a nested-scope symbol table;
 - multi-error semantic analysis;
 - TAC for expressions and control flow;
 - a reproducible Make-based build;
 - a 42-check automated regression suite; and
 - a Tkinter-based compiler interface.
-

2. Objectives

2.1 General objective

The general objective is to design and implement a complete MiniLang compiler front end, ending in Three Address Code, and to demonstrate the relationship between formal language rules and a maintainable software architecture.

2.2 Specific objectives

The project aims to:

1. implement all required token classes using Flex regular expressions;
2. track the line and column of each token;
3. implement a conflict-free MiniLang grammar using Bison;
4. recover from selected syntax errors at statement and block boundaries;
5. construct a meaningful AST during parsing;
6. traverse the AST through a reusable visitor interface;
7. maintain identifiers in nested lexical scopes;
8. detect undeclared names, redeclaration, scope violations, incompatible assignments, and invalid expressions;
9. infer expression types and preserve those results in the AST;
10. generate TAC for assignments, arithmetic, comparisons, logical expressions, branches, loops, and print statements;
11. report uniform diagnostics with helpful suggestions;
12. verify behavior using exact golden outputs;
13. expose compiler phases through a clear command-line interface; and
14. provide an optional responsive GUI for editing, compiling, inspecting, and testing programs.

2.3 Acceptance criteria

The implementation is considered successful when:

- make regenerates and builds the compiler from the Flex and Bison sources;
 - valid MiniLang programs return exit code 0 without diagnostics;
 - invalid programs return exit code 1 with the expected diagnostics;
 - TAC is produced only for syntactically and semantically valid programs;
 - every mandatory compiler module is independently inspectable;
 - all 42 automated checks pass; and
 - the GUI backend self-test passes tokens, AST, symbol-table, and TAC modes.
-

3. Language Specification

3.1 Data types

MiniLang contains three concrete data types.

Type	Description	Example literals
int	Integer value	0, 10, 42
float	Floating-point value	0.5, 1.25, 3.14
bool	Boolean value	true, false

An `int` value may be widened to `float`. Narrowing from `float` to `int` and mixing Boolean values with numeric values are rejected.

3.2 Lexical elements

Keywords are:

```
int float bool if else while print true false
```

Identifiers match:

```
[A-Za-z_][A-Za-z0-9_]*
```

The language supports integer and float literals, parentheses, braces, semicolons, whitespace, line comments, and block comments. Whitespace and comments are discarded by the scanner.

3.3 Operators

Category	Operators	Constraint	Result
Arithmetic	+, -, *, /	Numeric operands	int or float
Modulus	%	Both operands must be <code>int</code>	int
Relational	<, >, <=, >=	Numeric operands	bool
Equality	==, !=	Both numeric, or both <code>bool</code>	bool
Logical	&&,	Both operands must be <code>bool</code>	bool
Unary logical	!	Operand must be <code>bool</code>	bool
Unary arithmetic	-	Operand must be numeric	Operand type

3.4 Statements

MiniLang supports:

```
Declaration:  int x;
Assignment:   x = expression;
Print:        print expression;
If:           if (condition) statement
If-else:      if (condition) statement else statement
While:        while (condition) statement
Block:        { statement-list }
```

Declarations and assignments are separate. Declaration-initialization syntax such as `int x = 10;` is not supported by the implemented grammar.

3.5 Context-Free Grammar

The implemented grammar is summarized below. The Bison source uses one `expr` nonterminal and precedence declarations for expression ambiguity.

```
program      -> stmt_list

stmt_list    -> stmt_list stmt
              | empty

stmt          -> declaration
              | assignment
              | if_stmt
              | while_stmt
              | print_stmt
              | block

declaration  -> type_spec IDENT ';'

type_spec    -> 'int'
              | 'float'
              | 'bool'

assignment   -> IDENT '=' expr ';'

if_stmt      -> 'if' '(' expr ')' stmt
              | 'if' '(' expr ')' stmt 'else' stmt

while_stmt   -> 'while' '(' expr ')' stmt

print_stmt   -> 'print' expr ';'

block        -> '{' stmt_list '}'

expr         -> expr '||' expr
              | expr '&&' expr
              | expr '==' expr
              | expr '!=' expr
              | expr '<' expr
              | expr '>' expr
              | expr '<=' expr
              | expr '>=' expr
              | expr '+' expr
              | expr '-' expr
              | expr '*' expr
              | expr '/' expr
              | expr '%' expr
              | '!' expr
              | '-' expr
              | '(' expr ')'
              | IDENT
              | INT_LIT
              | FLOAT_LIT
              | 'true'
              | 'false'
```

3.6 Precedence and associativity

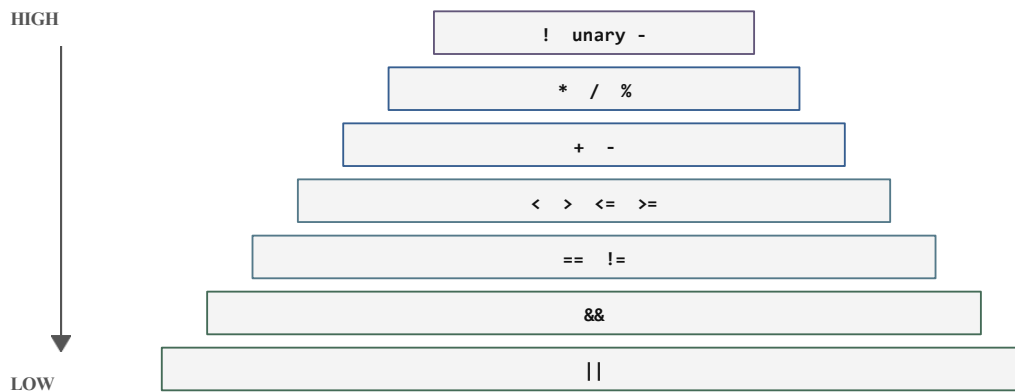


Figure 2. Operator-precedence ladder from highest to lowest binding.

Level, low to high	Operators	Associativity
1		left
2	&&	left
3	==, !=	left
4	<, >, <=, >=	left
5	+, -	left
6	*, /, %	left
7	!, unary -	unary precedence

The dangling `else` binds to the nearest unmatched `if`.

3.7 Semantic rules

- Every identifier must be declared before use.
- A name cannot be redeclared in the same scope.
- A declaration in an inner scope may shadow an outer declaration.
- A block-local name becomes inaccessible when its block ends.
- `if` and `while` conditions must be Boolean.
- Mixed `int/float` arithmetic produces `float`.
- Assigning `int` to `float` is allowed.
- Assigning `float` to `int` is rejected.
- `%` requires two integers.
- Logical operators require Boolean operands.
- Equality accepts numeric-to-numeric or Boolean-to-Boolean comparison.

3.8 Sample program

The main example follows the language defined in the project manual.

```
int x;
```

```
int y;  
bool flag;  
  
x = 10;  
y = 0;  
flag = true;  
  
while (x > 0) {  
    y = y + x;  
    x = x - 1;  
}  
  
if (flag == true) {  
    print y;  
} else {  
    print x;  
}
```

4. Compiler Architecture

4.1 Pipeline

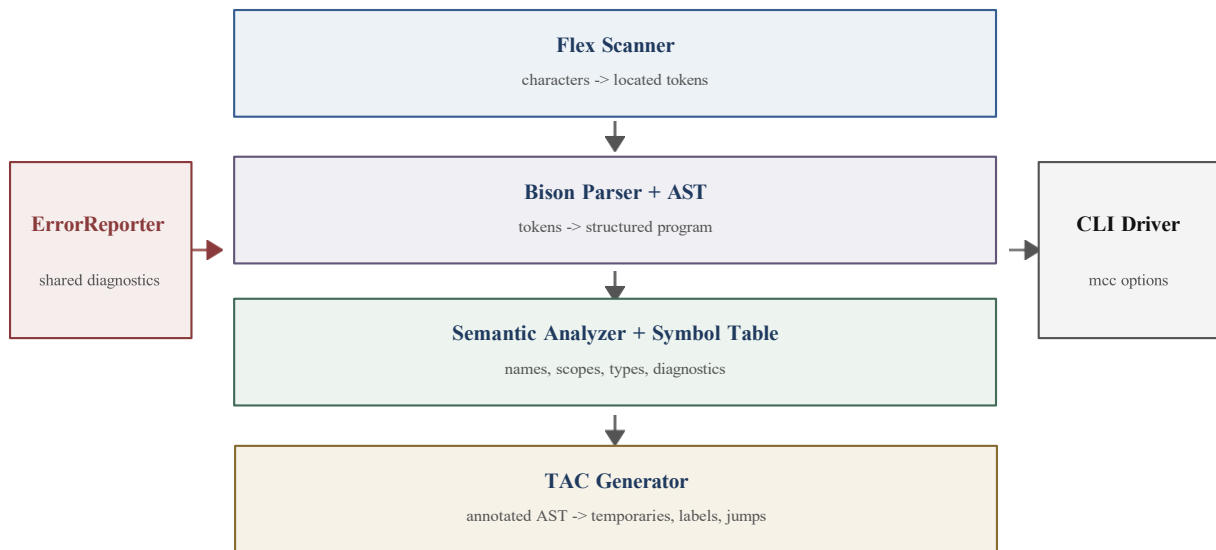


Figure 3. As-built compiler data flow and shared supporting modules.

The compiler uses a one-directional front-end pipeline.

```
Source file
|
v
Flex scanner
|
v
Bison parser and AST construction
|
v
Semantic analyzer and nested symbol table
|
v
Three Address Code generator
|
v
CLI output or .tac file
```

The parser constructs the AST directly; there is no separately stored concrete parse tree. The semantic analyzer writes inferred types into expression nodes. TAC generation occurs only after semantic analysis succeeds.

4.2 Module organization

Module	Main files	Responsibility
Common	source_location.hpp, type.hpp, error_reporter.*	Shared types and diagnostics
Lexer	src/lexer/lexer.l	Characters to tokens
Parser	src/parser/parser.y	Grammar validation and AST construction
AST	include/minilang/ast.hpp, src/ast/	Program representation and printer
Symbol table	include/minilang/symbol_table.hpp, src/symbol_table/	Nested declarations and lookup
Semantic	src/semantic/semantic_analyzer.cpp	Type and scope rules
TAC	include/minilang/tac*.hpp, src/tac/	Intermediate-code model and generation
Driver	src/main.cpp	CLI validation and pipeline coordination

4.3 Architectural patterns

Visitor pattern

Each AST node implements `accept(ASTVisitor&)`. `ASTPrinter`, `SemanticAnalyzer`, and `TACGenerator` implement the visitor interface. This keeps operations separate from node storage and avoids large node-kind switch statements.

Collecting diagnostics

Lexer, parser, and semantic analyzer report through a shared `ErrorReporter`. Diagnostics are stored, printed together, and used to determine the exit code. This permits multiple recoverable diagnostics.

Annotated AST

`ExprNode` contains a `Type` field. The semantic analyzer fills this field bottom-up. `AssignmentNode` records the resolved target type. TAC generation uses these annotations to decide, for example, whether an `int-to-float` cast is required.

Scoped hash tables

The active symbol table is a stack of hash maps. Lookup begins in the innermost scope. Exited scopes are archived for later symbol-table display.

4.4 Command-line interface

The compiler is built as `build/mcc`.

```
./build/mcc <source-file> [options]
```

Option	Purpose
--tokens	Print the scanner token stream
--ast	Parse and print the AST
--syntab	Analyze and print the symbol table
--tac	Generate TAC on standard output
-o <file>	Generate TAC and write it to a file
--help	Print usage information

Without an inspection option, the driver parses and semantically validates the program. It prints `Compilation successful`. when no error is present. TAC is printed only when requested by `--tac` or `-O`.

4.5 Desktop interface

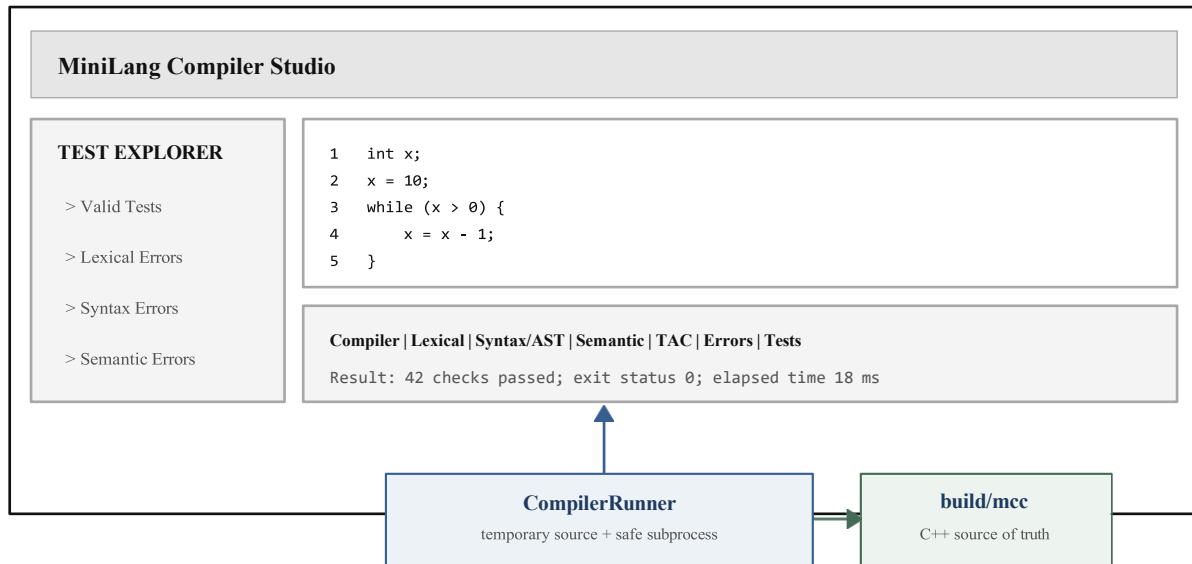


Figure 4. Compiler interface layout and subprocess boundary to build/mcc.

The optional GUI uses Python 3, Tkinter, and ttk. `run_gui.py` is its single entry point. The interface invokes `build/mcc` through a subprocess adapter; therefore, command-line and GUI compilation use exactly the same compiler.

The final interface provides:

- a Dark Modern IDE theme;
- a line-numbered MiniLang editor;
- syntax highlighting and automatic indentation;
- bracket matching, find/replace, go-to-line, undo/redo, and zoom;
- a repository-backed Test Explorer;
- a six-stage visual pipeline;
- structured lexical, AST, symbol-table, and TAC views;
- clickable diagnostics and editor markers;
- Compiler Output, Errors, Warnings, Console, Build Log, Test Suite, and Expected Output views;
- a cancellable 42-check regression dashboard;
- keyboard shortcuts and panel visibility controls; and
- persisted window and splitter positions.

Background threads execute build, compiler, and test processes. They send results to the Tk main thread through a queue, preventing unsafe widget access and keeping the interface responsive.

4.6 Repository structure

```
CC-Lab-Project-House_Compiler/
├─ include/minilang/      public C++ headers
├─ src/
│  ├─ lexer/             Flex source
│  ├─ parser/            Bison source
│  ├─ ast/               AST implementation
│  ├─ common/            shared diagnostics and token names
│  ├─ symbol_table/      nested symbol-table implementation
│  ├─ semantic/          semantic-analysis visitor
│  ├─ tac/               TAC model and generator
│  └─ main.cpp           CLI driver
├─ gui/                  Tkinter compiler studio
├─ examples/             representative MiniLang source
├─ tests/                valid, invalid, and golden-output cases
├─ scripts/run_tests.sh  shell regression runner
├─ docs/                 design, grammar, and report sources
├─ Makefile              build and test automation
├─ run_gui.py            GUI launcher and backend self-test
├─ GUI_SETUP.md          GUI installation instructions
└─ GUI_USER_GUIDE.md     GUI use and demonstration guide
```


5. Lexer Design

5.1 Responsibilities

The scanner is defined in `src/lexer/lexer.l`. It groups characters into tokens, discards non-semantic input, attaches locations, and reports lexical errors while continuing where possible.

5.2 Token categories

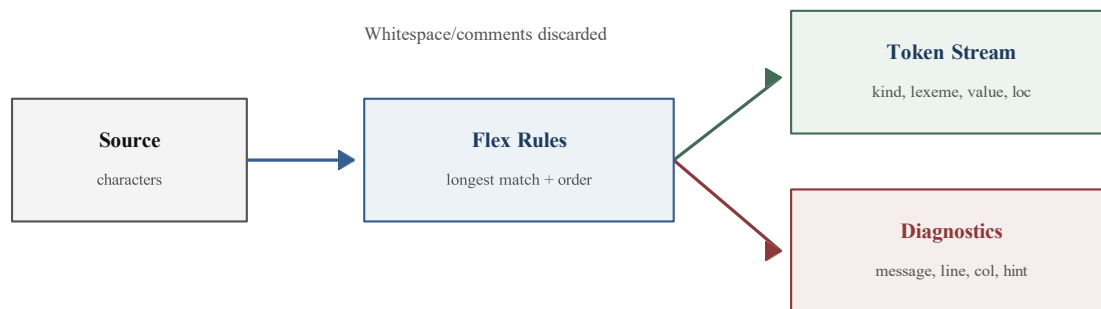


Figure 5. Scanner decision flow from source characters to tokens or diagnostics.

Category	Examples
Type keywords	<code>int, float, bool</code>
Control keywords	<code>if, else, while, print</code>
Boolean literals	<code>true, false</code>
Identifiers	<code>x, _count, total2</code>
Integer literals	<code>0, 25, 100</code>
Float literals	<code>0.5, 3.14</code>
Operators	arithmetic, relational, equality, logical, assignment
Delimiters	<code>{, }, (,), ;</code>

5.3 Longest match and rule ordering

Flex first chooses the longest matching rule and then uses rule order to break equal-length ties. Keyword rules are placed before the identifier rule, so `int` is a keyword. The longer text `integer` remains one identifier rather than the keyword `int` followed by another token.

Multi-character operators such as `<=`, `>=`, `==`, `!=`, `&&`, and `||` are placed before related single-character rules.

5.4 Location tracking

The scanner maintains `cur_line` and `cur_col`. `YY_USER_ACTION` records the start position of every matched lexeme and advances the column. A newline increments the line and resets the column to 1. The token location is shared with Bison through `yylloc`.

This mechanism supports diagnostics such as:

```
Lexical Error [line 3, col 1]: Invalid token '@'
--> hint: this character is not part of the MiniLang alphabet
```

5.5 Comments

// comments are discarded through the end of the current line. Block comments use an exclusive Flex start condition. The scanner remembers where a block comment began so an end-of-file error points to the opening /* rather than to an unrelated final character.

5.6 Malformed token handling

Dedicated rules appear before valid numeric rules and recognize:

- multiple decimal points, such as 1.2.3;
- a trailing decimal point without fractional digits;
- digit-starting identifiers, such as 123abc;
- single & and | characters; and
- unsupported characters.

These rules prevent misleading token splits and provide targeted hints.

5.7 Token inspection mode

--tokens prints location, token name, lexeme, and decoded value. This is a scanner-only mode and returns exit code 1 if any lexical error is collected.

6. Parser Design

6.1 Bison integration

The parser is defined in `src/parser/parser.y`. It uses `%union` values for literals, identifier strings, AST pointers, type values, and statement lists. The AST root is returned through a parse parameter instead of a global result pointer.

Bison's location type is replaced by the project's `SourceLocation`. Grammar actions can therefore use `@1`, `@2`, and related locations directly when constructing nodes.

6.2 Expression precedence

The grammar uses one `expr` nonterminal with precedence declarations. This approach keeps the grammar close to the language specification while producing the required parse tree. For example:

```
1 + 2 * 3
```

is interpreted as:

```
1 + (2 * 3)
```

6.3 Dangling else

Two precedence markers make `else` bind more strongly than reduction of an unmatched `if`. Therefore, `else` attaches to the nearest unmatched `if`, which is the standard language behavior.

6.4 Conflict checking

The Makefile invokes Bison with:

```
-Wall -Wcounterexamples -Werror=conflicts-sr -Werror=conflicts-rr
```

A shift/reduce or reduce/reduce conflict fails the build. This prevents hidden grammar ambiguity from being accepted accidentally.

6.5 Error messages

Bison's verbose messages are normalized into the common diagnostic format. Hints are selected for common mistakes, including:

- a missing semicolon;
- an unclosed parenthesis;
- a stray `else`;
- an invalid assignment target;
- a missing expression before `{`; and
- an unexpected end of file.

6.6 Recovery

Two recovery points are implemented:

```
stmt -> error ';'
block -> '{' stmt_list error '}'
```

The first skips to a statement terminator. The second recovers at the end of a block. Successfully parsed statements are kept, while the invalid recovered statement contributes no AST node.

6.7 AST construction

Every successful grammar reduction constructs or connects AST objects. Semicolons, parentheses, and grammar-only nodes do not appear in the AST. Bison destructors release discarded semantic values during error recovery.

7. Abstract Syntax Tree

7.1 Hierarchy

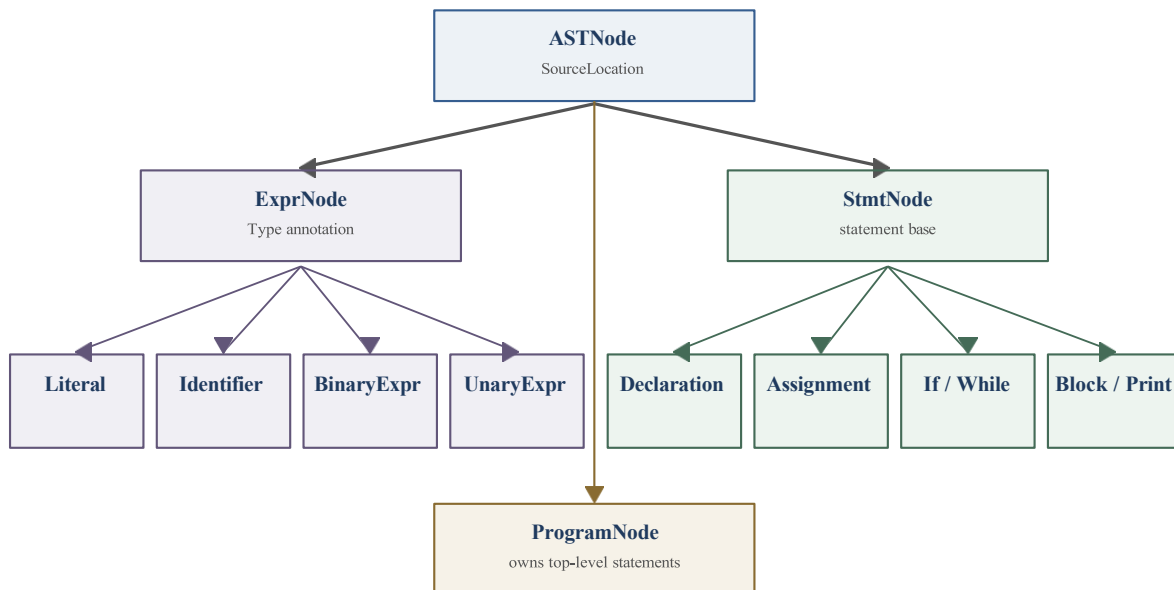


Figure 6. Simplified AST class hierarchy and ownership structure.

The AST is declared in `include/minilang/ast.hpp`.

```
ASTNode
├── ExprNode
│   ├── IntLiteralNode
│   ├── FloatLiteralNode
│   ├── BoolLiteralNode
│   ├── IdentifierNode
│   ├── BinaryExprNode
│   └── UnaryExprNode
├── StmtNode
│   ├── DeclarationNode
│   ├── AssignmentNode
│   ├── BlockNode
│   ├── IfNode
│   ├── WhileNode
│   └── PrintNode
└── ProgramNode
```

Every node records its source location. Expression nodes also contain a type annotation initialized to `Unresolved`. Assignment nodes contain a target type resolved during semantic analysis.

7.2 Ownership

Parent nodes own child pointers. Destructors recursively delete owned expressions, statements, branches, and statement lists. Deleting the root releases the complete tree.

Bison `%destructor` declarations cover values discarded during recovery, preventing leaks when parsing fails.

7.3 Visitor interface

`ASTVisitor` defines one `visit` operation for every concrete node. The three implemented visitors are:

Visitor	Role
ASTPrinter	Produce an indented structural view
SemanticAnalyzer	Resolve names, infer types, and check rules
TACGenerator	Emit intermediate instructions

7.4 AST output

The `--ast` option prints the tree immediately after parsing, before semantic analysis runs. It therefore shows node structure and source locations, not the later internal type annotations.

Example excerpt:

```

Program [1:1]
  Declaration 'x' : int [2:5]
  Declaration 'y' : int [3:5]
  Declaration 'flag' : bool [4:6]
  Assignment 'x' [6:1]
    IntLiteral 10 [6:5]
  While [10:1]
    Condition
      BinaryExpr '>' [10:10]
        Identifier 'x' [10:8]
        IntLiteral 0 [10:12]
```

The GUI parser converts this indented text into a collapsible AST tree. Nodes with locations can navigate back to the editor.

8. Symbol Table

8.1 Symbol record

Each symbol stores:

```
struct Symbol {  
    std::string name;  
    Type type;  
    int scopeLevel;  
    int declaredLine;  
    bool initialized;  
};
```

The initialization flag is updated after a valid assignment and is shown in the symbol-table output. The current compiler does not issue a separate uninitialized-read warning.

8.2 Scope representation

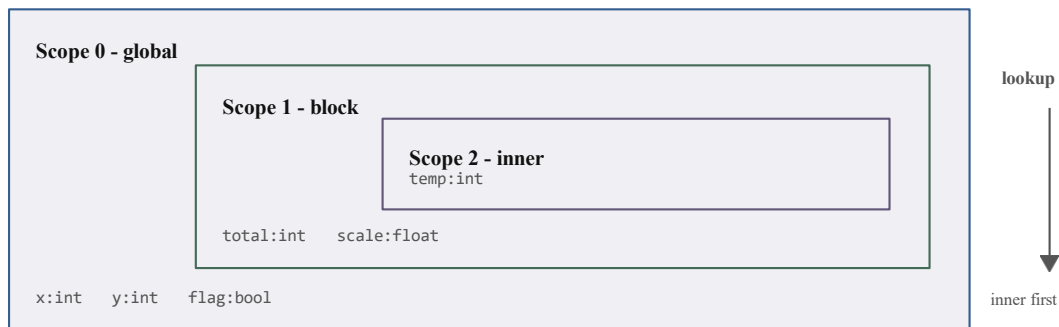


Figure 7. Nested scope stack with innermost-first identifier lookup.

Active scopes are stored as a vector of hash maps:

```
std::vector<std::unordered_map<std::string, Symbol>> scopes_;
```

The final element is the current scope. Entering a block pushes an empty map; leaving the block copies the completed scope into an archive and removes it from active lookup.

8.3 Operations

Operation	Behavior	Average complexity
insert	Insert into current scope	O(1)
lookupCurrentScope	Check current scope for redeclaration	O(1)
lookup	Search from inner scope to outer scope	O(depth)
enterScope	Push a new scope	O(1)
exitScope	Archive and pop the current scope	O(size of archived scope)

8.4 Shadowing and visibility

The same name may appear in different nested scopes. Lookup returns the nearest active declaration. A name declared in a block is no longer found after that block is exited. The semantic analyzer then reports the use as an undeclared or out-of-scope identifier.

8.5 Display

--symtab prints active and archived scopes. Example:

```
=== Symbol Table ===  
  
Scope Level 0 (global)  
-----  
NAME      TYPE    SCOPE  LINE  INITIALIZED  
x          int     0      2     yes  
y          int     0      3     yes  
flag       bool    0      4     yes  
  
Total symbols declared: 3
```

The GUI presents the same output as a filterable, source-aware table grouped by scope.

9. Semantic Analysis

9.1 Analysis process

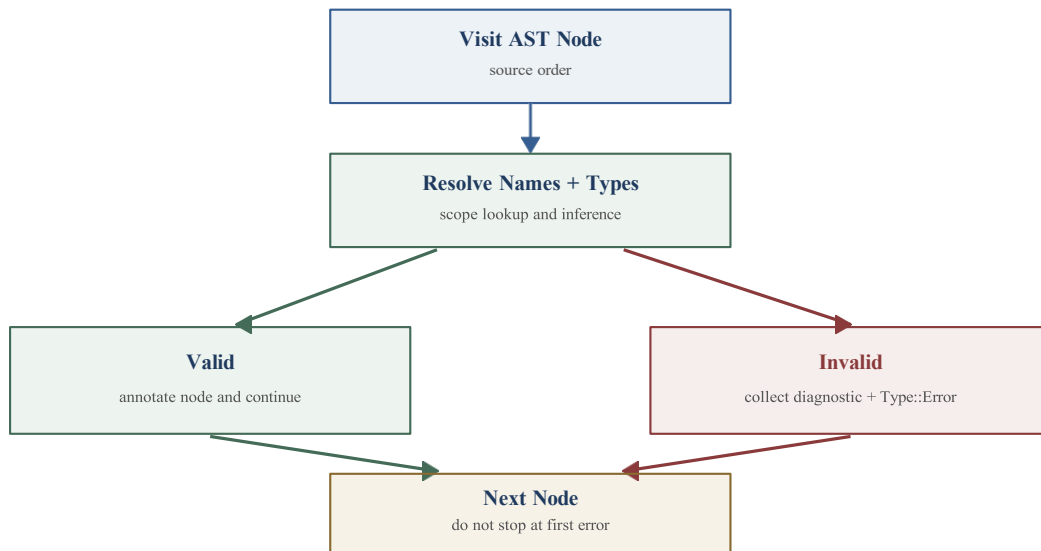


Figure 8. Semantic traversal, annotation, and multi-error collection.

The semantic analyzer visits statements in source order. Declarations are inserted when encountered; consequently, an identifier must be declared before use. A block creates a nested scope around its statements.

Expression analysis is bottom-up:

1. analyze child expressions;
2. read their types;
3. validate the operator rule;
4. store the resulting type in the parent expression; and
5. use `Type::Error` to suppress redundant cascades.

9.2 Detected errors

Error	Example using valid grammar
Undeclared variable	<code>print missing;</code>
Same-scope redeclaration	<code>int x; int x;</code>
Scope violation	declare x in a block, then use it after the block
Boolean-to-numeric assignment	<code>int x; x = true;</code>
Numeric-to-Boolean assignment	<code>bool ready; ready = 1;</code>
Narrowing assignment	<code>int x; x = 3.14;</code>
Invalid arithmetic	<code>int x; x = true + 1;</code>
Invalid modulus	<code>float f; f = 4.5 % 2;</code>
Invalid logical expression	<code>bool b; b = 1 && 2;</code>
Invalid equality	<code>bool b; b = 1 == true;</code>
Invalid condition	<code>int x; if (x) { print x; }</code>

9.3 Assignment compatibility

Target	Accepted source	Rejected source
int	int	float, bool
float	float, int	bool
bool	bool	int, float

9.4 Multi-error behavior

The analyzer does not stop after its first independent semantic error. The semantic multi-error test contains four problems:

```
int a;  
int a;  
bool flag;  
a = true;  
flag = a;  
print z;
```

The output contains four diagnostics: redeclaration, two incompatible assignments, and one undeclared identifier. The GUI displays four rows and four corresponding editor markers.

9.5 Suppression of later phases

If parsing failed, semantic analysis is not started. If semantic analysis added errors, TAC generation is not started. This prevents invalid programs from producing misleading intermediate code.

10. Intermediate Code Generation

10.1 Purpose

Three Address Code is a linear representation in which each instruction contains at most one operator. It separates source-language structure from machine-specific concerns and is suitable for later optimization or backend translation, although those later phases are outside this project.

10.2 Instruction forms

The TAC model supports:

```
x = literal
x = y
x = y op z
x = op y
x = (float) y
ifTrue x goto L
ifFalse x goto L
goto L
L:
print x
```

Temporary variables are named **t1**, **t2**, and so on. Labels are named **L1**, **L2**, and so on. Counters start fresh for each compilation.

10.3 Expressions

The generator recursively evaluates operands and emits an instruction for the current operator.

Source:

```
c = a + b * 2;
```

TAC:

```
t1 = b * 2
t2 = a + t1
c = t2
```

10.4 Assignment widening

An integer assigned to a float target generates an explicit cast:

```
int n;
float f;
n = 4;
f = n;
```

```
n = 4
t1 = (float) n
f = t1
```

10.5 Branches

An **if-else** statement uses an else label and an end label:

```
[condition code]
ifFalse condition goto L_else
[then code]
goto L_end
L_else:
[else code]
L_end:
```

10.6 Loops

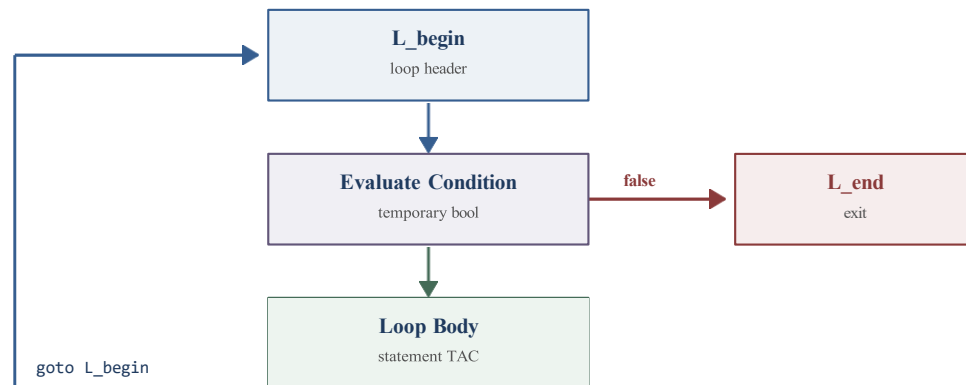


Figure 9. Control-flow graph used to lower a while loop into TAC.

A **while** statement uses a beginning label, a false exit, and a back jump:

```

L_begin:
[condition code]
ifFalse condition goto L_end
[body code]
goto L_begin
L_end:
  
```

The condition code is inside the loop so it is recomputed on each iteration.

10.7 Short-circuit logical expressions

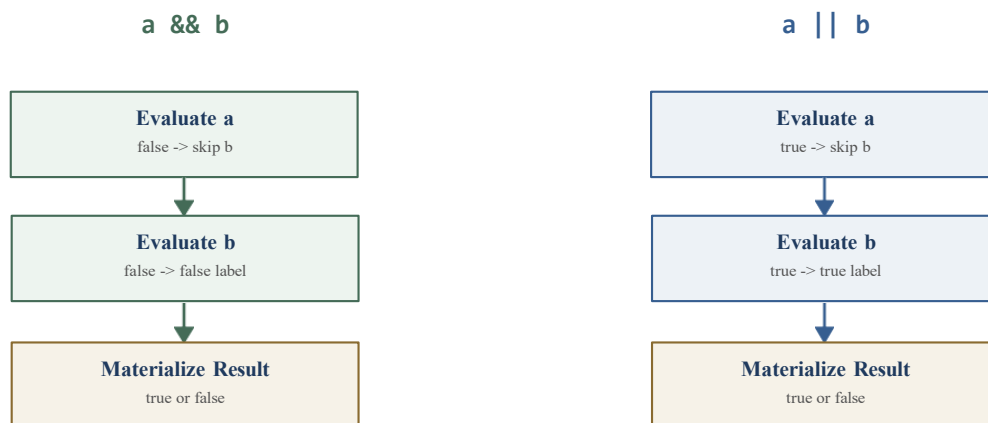


Figure 10. Short-circuit evaluation paths for logical AND and OR.

&& and **||** are generated through conditional jumps rather than eager binary instructions.

For **a && b**, the right side is skipped when **a** is false:

```

ifFalse a goto L_false
ifFalse b goto L_false
t1 = true
goto L_end
L_false:
t1 = false
  
```

```
L_end:
```

For `a || b`, the right side is skipped when `a` is true:

```
ifTrue a goto L_true
ifTrue b goto L_true
t1 = false
goto L_end
L_true:
t1 = true
L_end:
```

10.8 Complete sample output

The sample program generates:

```
; === Three Address Code: examples/sample.mc ===

    x = 10
    y = 0
    flag = true
L1:
    t1 = x > 0
    ifFalse t1 goto L2
    t2 = y + x
    y = t2
    t3 = x - 1
    x = t3
    goto L1
L2:
    t4 = flag == true
    ifFalse t4 goto L3
    print y
    goto L4
L3:
    print x
L4:
```

Ten TAC programs are compared character-for-character with golden `.tac` files.

11. Challenges

11.1 Grammar conflicts and recovery

Expression ambiguity and dangling `else` can introduce shift/reduce conflicts. Precedence declarations resolved the expression grammar, while a separate precedence pair implemented nearest-`if` binding. Recovery rules also had to synchronize at useful tokens without creating new ambiguity.

The build treats conflicts as errors and enables Bison counterexamples. This turned parser ambiguity into a visible build-time problem rather than a hidden runtime behavior.

11.2 Precise locations across Flex and Bison

Line tracking alone was insufficient for editor navigation and useful diagnostics. Flex and Bison needed to share the same location representation. The solution combined `YY_USER_ACTION`, custom newline handling, `%locations`, and a custom `YYLTYPE`.

11.3 Malformed-number recovery

Without dedicated patterns, an input such as `1.2.3` could be split into several apparently valid tokens and cause a confusing parser error. Longer malformed-input rules were placed before the valid numeric rules so the scanner could diagnose the real lexical problem.

11.4 AST memory ownership

The Bison parser creates raw AST pointers in semantic actions. Both normal tree destruction and error-recovery discards had to be handled. Virtual destructors, parent ownership, and Bison `%destructor` declarations provided a consistent ownership policy.

11.5 Header dependency tracking

Changing an AST class layout while keeping stale object files can produce binary incompatibility and heap corruption. The Makefile uses `-MMD -MP` and includes generated dependency files so changes to public headers recompile all affected objects. A clean build remains the recommended final verification.

11.6 Cross-platform golden outputs

TAC headings contain the source path. Windows and Linux use different path separators, and line endings may also differ. Repository tests use stable relative forward-slash paths. The Python regression runner normalizes line endings while retaining exact semantic content.

11.7 Responsive GUI execution

Running `make`, four compiler modes, or 42 regression checks directly in the Tk main loop would freeze the interface. The GUI therefore executes those operations in worker threads and returns result objects through a queue. Widget updates occur only on the Tk main thread.

11.8 Structured output without changing the compiler

The CLI was already the verified source of truth. The GUI needed tables and trees without modifying the compiler output contract. Dedicated Python parsers convert the existing token, AST, symbol-table, and TAC text into structured views while preserving raw output for copying and comparison.

12. Testing

12.1 Testing strategy

Testing combines exit-code checks, absence or presence of diagnostics, and exact golden-output comparison.

- A valid compilation must return 0 and produce no standard-error output.
- An invalid test must return 1 and match its `.err` file exactly.
- A TAC test must return 0 and match its `.tac` file exactly.
- The GUI backend self-test must successfully exercise all four inspection modes.
- The GUI regression dashboard must reproduce the complete 42-check suite.

12.2 Accurate test inventory

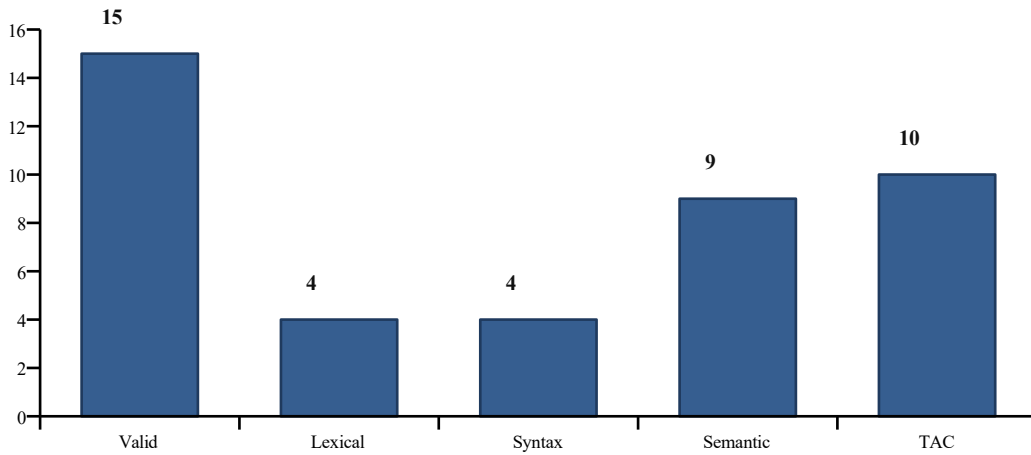


Figure 11. Distribution of all 42 automated checks.

The repository contains 32 distinct MiniLang source programs. Ten valid TAC programs are each tested twice: once for successful compilation and once for exact TAC output. Therefore, there are 42 checks, not 42 distinct programs.

Check category	Count
Valid compilation in <code>tests/valid</code> and <code>examples</code>	15
Lexical diagnostic comparisons	4
Syntax diagnostic comparisons	4
Semantic diagnostic comparisons	9
TAC golden-output comparisons	10
Total	42

The 17 invalid checks consist of 4 lexical, 4 syntax, and 9 semantic cases.

12.3 Coverage

Valid programs cover:

- complete token recognition;
- nested blocks and dangling `else`;
- scope creation and legal shadowing;
- valid semantic combinations;
- arithmetic and precedence;
- integer-to-float casts;
- unary operations;
- relational and logical expressions;
- modulus;
- nested branches;
- nested loops;
- `while`; and
- the manual-aligned sample program.

Invalid programs cover:

- unsupported characters;
- invalid identifiers and near-miss operators;
- malformed numbers;
- unterminated comments;
- missing semicolons;
- unbalanced parentheses;
- stray `else`;
- multiple syntax errors;
- undeclared variables;
- redeclaration;
- scope violations;
- type mismatch;
- invalid arithmetic;
- invalid logical expressions;
- invalid equality;
- invalid condition types; and
- multiple semantic errors.

12.4 Test commands

```
make clean
make
make test
```

Direct execution is also available:

```
./scripts/run_tests.sh
```

GUI verification uses:

```
python3 -m compileall -q gui run_gui.py
python3 run_gui.py --self-test
python3 run_gui.py
```

Inside the GUI, Ctrl+T runs the dashboard suite.

12.5 Verified results

The command-line suite produced:

```
-----
42 passed, 0 failed
```

The backend self-test produced successful results for:

```
PASS tokens
PASS ast
PASS symtab
PASS tac
GUI backend self-test passed.
```

The integrated GUI dashboard also completed with 42 passed and 0 failed. The semantic multi-error scenario produced four diagnostics and four editor markers.

12.6 Reproducibility and repository cleanliness

Before a final commit or submission archive:

```
rm -rf -- __pycache__gui/__pycache__
rm -f -- output.tac
git status --short
```

Generated parser, scanner, object, dependency, and executable files belong in `build/`. The test script must retain executable permission on Linux.

13. Conclusion

13.1 Achievement summary

The project successfully implements all six mandatory compiler phases. Flex recognizes the complete MiniLang token set and provides exact locations. Bison validates the grammar, resolves precedence and dangling `else`, and builds the AST. The symbol table models nested lexical scopes. The semantic analyzer enforces declaration, scope, assignment, expression, comparison, logical, and condition rules. The TAC generator translates validated programs into a clear intermediate representation with temporaries, labels, casts, branches, loops, and short-circuit logic.

The architecture keeps responsibilities separated. The AST is shared without coupling it to semantic analysis or code generation. Diagnostics are collected through one interface. The CLI remains the authoritative compiler. The GUI is a separate, responsive client that presents compiler output in an accessible form.

13.2 Learning outcomes

The implementation provided practical experience with:

- regular-expression design and longest-match behavior;
- LR parsing and conflict resolution;
- syntax-error recovery;
- AST ownership and the visitor pattern;
- nested symbol lookup and shadowing;
- static type checking;
- propagation and suppression of semantic errors;
- control-flow lowering into TAC;
- short-circuit Boolean generation;
- build dependency management;
- golden-file regression testing; and
- thread-safe desktop integration with a command-line backend.

13.3 Limitations

The implemented system intentionally stops at TAC. It does not include:

- arrays;
- functions and return statements;
- `for`, `do-while`, or `switch`;
- increment/decrement operators;
- constant folding;
- dead-code elimination;
- assembly or machine-code generation;

- register allocation;
- Graphviz AST export;
- compiler-native uninitialized-variable warnings; or
- compile-time division-by-zero warnings.

These are optional future extensions and are not required for the completed base compiler.

13.4 Future work

The safest future improvements are independent visitors or new compiler passes, such as Graphviz AST export and constant folding. Any language syntax extension should be documented separately so the instructor-defined base grammar remains stable.

The GUI could later gain detachable docking if migrated to a toolkit with native dock support. The current Tkinter implementation instead provides resizable and independently hideable panels.

13.5 Final result

The accurate final statement is:

The MiniLang compiler implements all mandatory compiler phases and an optional Tkinter desktop interface. It is verified by 42 passing checks over 32 distinct MiniLang programs. TAC is the final generated representation, and no unimplemented bonus feature is claimed as complete.

14. References

1. Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. **Compilers: Principles, Techniques, and Tools**. 2nd edition. Addison-Wesley, 2006.
2. Levine, J. R., Mason, T., and Brown, D. **lex & yacc**. 2nd edition. O'Reilly Media, 1992.
3. Free Software Foundation. **GNU Bison Manual**.
4. Free Software Foundation. **Flex: The Fast Lexical Analyzer Manual**.
5. ISO/IEC. **Programming Languages - C++**, ISO/IEC 14882.
6. Python Software Foundation. **Python Standard Library Documentation: tkinter - Python interface to Tcl/Tk**.
7. Metropolitan University, Department of Computer Science and Engineering. **Compiler Construction Lab Project Manual**, 2026.
8. MiniLang project repository. Source code, tests, architecture notes, and implementation documentation.

Appendix A - Build, CLI, and GUI Command Reference

The following commands reproduce the build, compiler inspection modes, automated verification, and desktop launch.

```
sudo apt update
sudo apt install -y build-essential flex bison make python3 python3-tk

make clean
make
./build/mcc examples/sample.mc
./build/mcc examples/sample.mc --tokens
./build/mcc examples/sample.mc --ast
./build/mcc examples/sample.mc --symtab
./build/mcc examples/sample.mc --tac
./build/mcc examples/sample.mc -o output.tac

python3 -m compileall -q gui run_gui.py
python3 run_gui.py --self-test
./scripts/run_tests.sh
python3 run_gui.py
```

Appendix B - Complete Automated Check Inventory

Category	Source	Expected result
Valid	tests/valid/lexer_all_tokens.mc	exit 0, empty stderr
Valid	tests/valid/nested_blocks_dangling_else.mc	exit 0, empty stderr
Valid	tests/valid/nested_scopes_syntab.mc	exit 0, empty stderr
Valid	tests/valid/semantic_valid.mc	exit 0, empty stderr
Valid	tests/valid/tac_arithmetic.mc	exit 0, empty stderr
Valid	tests/valid/tac_cast_unary.mc	exit 0, empty stderr
Valid	tests/valid/tac_complete.mc	exit 0, empty stderr
Valid	tests/valid/tac_float.mc	exit 0, empty stderr
Valid	tests/valid/tac_if_else.mc	exit 0, empty stderr
Valid	tests/valid/tac_logical.mc	exit 0, empty stderr
Valid	tests/valid/tac_modulus.mc	exit 0, empty stderr
Valid	tests/valid/tac_nested_if.mc	exit 0, empty stderr
Valid	tests/valid/tac_nested_loops.mc	exit 0, empty stderr
Valid	tests/valid/tac_while.mc	exit 0, empty stderr
Valid	examples/sample.mc	exit 0, empty stderr
Lexical	tests/invalid/lexical/bad_identifier_and_ops.mc	exit 1, exact .err
Lexical	tests/invalid/lexical/invalid_token.mc	exit 1, exact .err
Lexical	tests/invalid/lexical/malformed_numbers.mc	exit 1, exact .err
Lexical	tests/invalid/lexical/unterminated_comment.mc	exit 1, exact .err
Syntax	tests/invalid/syntax/missing_semicolon.mc	exit 1, exact .err
Syntax	tests/invalid/syntax/multiple_errors.mc	exit 1, exact .err
Syntax	tests/invalid/syntax/stray_else.mc	exit 1, exact .err
Syntax	tests/invalid/syntax/unbalanced_paren.mc	exit 1, exact .err
Semantic	tests/invalid/semantic/invalid_condition.mc	exit 1, exact .err
Semantic	tests/invalid/semantic/invalid_equality.mc	exit 1, exact .err
Semantic	tests/invalid/semantic/invalid_expression_arith.mc	exit 1, exact .err
Semantic	tests/invalid/semantic/invalid_expression_logical.mc	exit 1, exact .err
Semantic	tests/invalid/semantic/multiple_errors.mc	exit 1, exact .err
Semantic	tests/invalid/semantic/redeclaration.mc	exit 1, exact .err
Semantic	tests/invalid/semantic/scope_violation.mc	exit 1, exact .err
Semantic	tests/invalid/semantic/type_mismatch.mc	exit 1, exact .err
Semantic	tests/invalid/semantic/undeclared_variable.mc	exit 1, exact .err
TAC	tests/valid/tac_arithmetic.mc	exit 0, exact .tac
TAC	tests/valid/tac_cast_unary.mc	exit 0, exact .tac
TAC	tests/valid/tac_complete.mc	exit 0, exact .tac
TAC	tests/valid/tac_float.mc	exit 0, exact .tac
TAC	tests/valid/tac_if_else.mc	exit 0, exact .tac
TAC	tests/valid/tac_logical.mc	exit 0, exact .tac
TAC	tests/valid/tac_modulus.mc	exit 0, exact .tac
TAC	tests/valid/tac_nested_if.mc	exit 0, exact .tac
TAC	tests/valid/tac_nested_loops.mc	exit 0, exact .tac
TAC	tests/valid/tac_while.mc	exit 0, exact .tac

The inventory contains 42 rows because ten TAC sources appear once as valid compilations and once as golden-output comparisons. There are 32 distinct source programs.

Appendix C - Compiler Studio Keyboard Shortcuts

Shortcut	Action
Ctrl+N	New source buffer
Ctrl+O	Open source file
Ctrl+S	Save
Ctrl+Shift+S	Save as
F5	Run full compiler pipeline
Ctrl+B	Build compiler
Ctrl+T	Run all 42 checks
Ctrl+1	Lexical analysis
Ctrl+2	Syntax and AST
Ctrl+3	Semantic analysis and symbols
Ctrl+4	Generate TAC
Ctrl+F	Find
Ctrl+H	Replace
Ctrl+G	Go to line
Ctrl++ / Ctrl+-	Zoom editor
Ctrl+0	Reset editor zoom
Ctrl+Shift+E	Toggle Test Explorer
Ctrl+J	Toggle output panel
F11	Toggle full screen
Escape	Leave full screen